



Bahria University, Islamabad

Department of Software Engineering

Formals Methods in Software Engineering (Fall-2025)

Teacher: RAFIA MUMTAZ

1.Student: Malik Shajar Hussain

Enrollment: 01-131232-043

2.Student: Abdul Wahab

Enrollment:01-131232-007

Date:11/8/2025

Assignment-3

ASSIGNMENT 3 – FORMAL METHODS (Z SPECIFICATION)

QUESTION 1 – CINEMA TICKET BOOKING SYSTEM

Explanation

This system models an **online cinema ticket booking system** where only registered users are allowed to reserve seats for movie shows. A cinema hall has a fixed set of seats for each show, and once a seat is booked for a specific show, it must not be available to any other user for the same show. However, the same seat can be booked again for a different show.

The main purpose of this specification is to formally ensure that:

- No seat is double-booked for the same show
- Only registered users are allowed to book tickets
- Bookings can be cancelled before the show time
- A proper record of all bookings is maintained

Basic Type Definitions

[USER, SHOW, SEAT, BOOKINGID, TIME]

State Schema

UniversitySystem
students : $\mathbb{P}$ STUDENT
courses : $\mathbb{P}$ COURSE
enrolled : STUDENT $\rightarrow$ $\mathbb{P}$ COURSE
completed : STUDENT $\rightarrow$ $\mathbb{P}$ COURSE
prerequisites : COURSE $\rightarrow$ $\mathbb{P}$ COURSE
capacity : COURSE $\rightarrow$ $\mathbb{N}$

 $\forall s : \text{STUDENT}; c : \text{COURSE} \bullet$
 $c \in \text{enrolled}(s) \Rightarrow \# \text{enrolled} \sim [\{c\}] \leq \text{capacity}(c)$

Explanation

- `students` stores all students registered in the university.
- `courses` represents the set of all available courses.
- `enrolled(s)` gives the set of courses a student is currently taking.
- `completed(s)` stores courses already passed by the student.
- `prerequisites(c)` defines which courses must be completed before enrolling in course `c`.
- `capacity(c)` limits the maximum number of students allowed in a course.

The invariant guarantees that the number of students enrolled in any course never exceeds its capacity. This constraint must hold in every valid state of the system.

Initialize System

```
InitCinemaSystem
CinemaSystem
registeredUsers = ∅
shows = ∅
bookings = ∅
```

Register User

```
RegisterUser
ΔCinemaSystem
user? : USER
user? ∉ registeredUsers
registeredUsers' = registeredUsers ∪ {user?}
```

Book Ticket (Normal Case)

```
BookTicket
ΔCinemaSystem
user? : USER
show? : SHOW
seats? :  $\mathbb{P}$  SEAT
bid! : BOOKINGID

user? ∈ registeredUsers
show? ∈ shows
```

```

seats?  $\subseteq$  seats(show?)
 $\forall b : \text{BOOKINGID} \bullet$ 
  let (_, s, ss) == bookings(b)  $\bullet$ 
    s  $\neq$  show?  $\vee$  ss  $\cap$  seats? =  $\emptyset$ 

bookings' = bookings  $\cup$  { bid!  $\mapsto$  (user?, show?, seats?) }

```

Cancel Booking

```

CancelBooking
 $\Delta$ CinemaSystem
bid? : BOOKINGID
currentTime? : TIME

bid?  $\in$  dom bookings
currentTime? < showTime(fst(snd(bookings(bid?))))

bookings' = {bid?}  $\Leftarrow$  bookings

```

Error Handling (Robust Specification)

Error Type

```
REPORT ::= ok | notRegistered | seatUnavailable | showPassed
```

Success

```

Success
result! : REPORT
result! = ok

```

Seat Unavailable

```

SeatUnavailable
 $\Xi$ CinemaSystem
result! : REPORT
result! = seatUnavailable

```

Robust Booking

$$RBookTicket \triangleq (BookTicket \wedge Success) \vee SeatUnavailable$$

QUESTION 2 – UNIVERSITY COURSE REGISTRATION SYSTEM

Explanation

This system represents a **centralized university course registration system**. Students can register for courses, drop courses, and progress through their degree by completing prerequisites. Administrators manage students, courses, and course capacities.

The goal of this specification is to formally capture academic rules such as:

- Only registered students can enroll in courses
- A student must complete all prerequisites before enrolling
- Each course has a maximum capacity that must not be exceeded
- Students are free to drop courses they are already enrolled in

Basic Types

$$[STUDENT, COURSE]$$

State Schema

```
UniversitySystem
students      :  $\mathbb{P}$  STUDENT
courses      :  $\mathbb{P}$  COURSE
enrolled      : STUDENT  $\rightarrow$   $\mathbb{P}$  COURSE
completed     : STUDENT  $\rightarrow$   $\mathbb{P}$  COURSE
prerequisites : COURSE  $\rightarrow$   $\mathbb{P}$  COURSE
capacity      : COURSE  $\rightarrow$   $\mathbb{N}$ 

 $\forall s : STUDENT; c : COURSE \bullet$ 
   $c \in enrolled(s) \Rightarrow \#enrolled-\{c\} \leq capacity(c)$ 
```

Add Student

```
AddStudent
ΔUniversitySystem
s? : STUDENT
s? ∉ students
students' = students ∪ {s?}
```

Add Course

```
AddCourse
ΔUniversitySystem
c? : COURSE
cap? : N
c? ∉ courses
courses' = courses ∪ {c?}
capacity' = capacity ⊕ {c? ↦ cap?}
```

Register Course

```
RegisterCourse
ΔUniversitySystem
s? : STUDENT
c? : COURSE

s? ∈ students
c? ∈ courses
prerequisites(c?) ⊆ completed(s?)
#enrolled~[{c?}] < capacity(c?)

enrolled' = enrolled ⊕ {s? ↦ enrolled(s?) ∪ {c?}}
```

Drop Course

```
DropCourse
ΔUniversitySystem
```

$s? : \text{STUDENT}$

$c? : \text{COURSE}$

$c? \in \text{enrolled}(s?)$

$\text{enrolled}' = \text{enrolled} \oplus \{s? \mapsto \text{enrolled}(s?) \setminus \{c?\}\}$

QUESTION 3 – REASONING ABOUT LOOPS

This question checks whether you understand **loop invariants** and how they are used to reason about correctness

Part 1 – Computing the Sum $1 + 2 + \dots + n$

Question

Write a loop to compute $\text{sum} = 1 + 2 + \dots + n$ and show that it is correct using a loop invariant.

Answer

Loop

```
sum := 0;
i := 1;
while i ≤ n do
  sum := sum + i;
  i := i + 1;
end while
```

Loop Invariant

$\text{sum} = 1 + 2 + \dots + (i - 1)$

Explanation At the start of the loop, no numbers have been added yet. Since i is equal to 1 and sum is 0, the invariant holds because the sum of numbers up to $i - 1 = 0$ is 0.

During each iteration, the current value of i is added to sum , and then i is increased by 1. This means that after one complete iteration, sum now contains the sum of all values from 1 up to the new $(i - 1)$. Hence, the invariant remains true throughout the loop.

When the loop terminates, the condition $i \leq n$ becomes false, which implies $i = n + 1$. Substituting this value into the invariant gives:

$$\text{sum} = 1 + 2 + \dots + n$$

Therefore, the loop correctly computes the required sum.

Part 2 – Tracing the Loop

Question

Given the following loop, find the value of j when the loop exits and describe the relationship between i and j .

```
i := 0;
j := 0;
while i ≤ n do
  i := i + 1;
  j := j + 2;
end while
```

Answer

To understand the behavior of the loop, we trace the values of i and j for the first few iterations.

At the start, both i and j are initialized to 0. In each iteration, i is increased by 1 and j is increased by 2. This creates the following sequence of values:

$$(i, j) = (0, 0), (1, 2), (2, 4), (3, 6), \dots$$

From this trace, it can be seen that j increases twice as fast as i . The loop terminates when i becomes $n + 1$, because the condition $i \leq n$ is no longer satisfied. At this point, the value of j is:

$$j = 2(n + 1)$$

Part 3 – Loop Invariant

Question

State a suitable loop invariant for the loop in Part 2 and justify it.

Answer

A suitable loop invariant for the given loop is:

$$j = 2 \times i$$

Explanation Initially, both i and j are equal to 0, so the invariant holds. During each iteration of the loop, i is increased by 1 and j is increased by 2, which preserves the relationship $j = 2 \times i$. This relationship continues to remain true after every iteration.

When the loop terminates, i has the value $n + 1$. Substituting this value into the invariant explains the final result of the loop:

$$j = 2(n + 1)$$

Thus, the invariant correctly describes the behavior of the loop and justifies the final value of j .